

Figure 1: Setting the environment variable

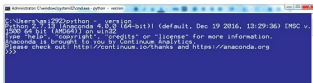


Figure 2: Checking the correct installation of Python and its version

text editor, which can be downloaded from <https://www.sublimetext.com/3>.

The cliché of programming languages is 'Hello World'; so let's get started by printing it in the console. Open the command prompt, type Python and execute it. Now run the command:

```
print ("Hello world").
```

This will print 'Hello World' in the console as shown in Figure 3. Python, being a scripting language, can also be executed by writing a group of individual statements, which make a single code block called a 'suite' in a file, and can be saved with the .py extension.

Now let's learn about the building blocks of Python, like the identifiers, keywords, user input processes, comments, etc. Identifiers are the names given to classes, variables, functions, etc. These are case sensitive and start with the letters A-Z, a-z or an underscore. There are some naming conventions in Python, which are listed below:

- An identifier starting with an underscore means that it's private, and a double underscore means it is strongly private.
- The class name must start with upper case. Python has some reserved words called keywords which have special meanings. Examples of keywords are *and*, *finally*,

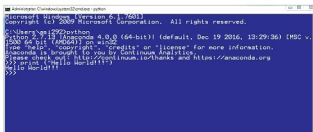


Figure 3: Getting started with printing 'Hello World'

Class, *def*, *if*, *else*, *elif*, etc. Unlike Java or C, which use braces to wrap lines of code, Python uses indentation to denote the block of code that is tightly mapped. In Python, statements usually end with a new line, but if the statement gets carried to the new line, then one uses the '\n' backslash to show the continuation of the line. A single line comment is denoted by '#' and a multi-line comment can be wrapped in triple quotes.

The data types used in Python are numbers, strings, lists, tuples and the dictionary. The number data type stores the numeric value and is immutable in nature, i.e., upon changing the value, a new object is created and the older one is kept for automatic garbage collection. For example, `variable = 10`. The single quote or double quotes are used to denote the string. For example, `varA = 'Hello World'`, `varB = "Python is cool"`. Python does not support character types, but treats the characters as strings of length one. Coming to the list, it consists of comma-separated values which could be of any type and it is represented as `[...]`. All values are enclosed between '[' and ']'. A list object is a mutable data type, which means it can't be hashed, and modifying an existing list will not result in a new list object—the memory address will not be changed either. For example: `def_list = [1, 2, "1", "100", "Python", "Anne"]`.

The next is a tuple, which has comma-separated values that could be of any type, enclosed between '(' and ')'. This, too, is an immutable data type and can be used as the key in a dictionary. For example: `def_tuple = (1, 2, "1", "100", "Python", "Anne")`. The dictionary in Python contains the key value pair in which the key is immutable, and the value can be anything; the key value is separated by a colon ':'. Dictionaries work on the hashing principle. For example: `dictB = dict()`. Let us now create a new file in the Sublime text editor with the following commands:

```
# An Empty Dict
dictA = dict() # same as dictA = {}
dictB = dict() # same as dictB = {}
# Adding values to it
for i in [1, 3, 2, 4, 8, 9, 5, 6, 7]:
    dictB[i] = 1
# Adding values to a dict

# Let us use a simple for loop to add values to a dict
for i in xrange(10):
    dictA[i] = i + 10
print dictA, "\n\n", dictB, "\n\n"

...
Define a simple list with multiple datatypes
...
def_list = [1, 2, "1", "100", "Python", "Anne", "A!@345<>()", True,
False, {1:100, 2:200, 3:300}, range(10)]
```